

Qwen3-Coder-Next Tier 3

Part 11 Detailed Implementation Specification

Context Compression

Engineering specification for the subsystem that selects, trims, summarizes, and packages repository context for downstream agents.

This document bridges the architecture reference and the detailed implementation work that follows.

Part	11
Subsystem	Context Compression
Primary role	Keep prompts within budget without losing the facts the agent needs
Upstream / downstream	Consumes repository intelligence and memory state; feeds planner, coding, testing, review, evaluation, and recovery

Contract	Purpose	Required fields	Notes
ContextRequest	Canonical retrieval input	task_id, agent_role, intent, token_budget, seed_sources, freshness_window	Must be deterministic
EvidenceChunk	Atomic source unit	source_id, location, excerpt, score, recency, provenance	Excerpt can be raw or summarized
ContextPackage	Final handoff payload	package_id, task_id, selected_chunks, compressed_brief, budget_used, omissions, provenance_map	Consumed by downstream agents
BudgetReport	Accounting output	requested_budget, used_budget, reserved_budget, dropped_sources, reason_codes	Used for validation and telemetry
CompressionConfig	Policy definition	max_tokens, min_source_coverage, summary_style, dedupe_threshold, freshness_priority	Versioned and immutable per run

normalize request -> resolve sources -> retrieve candidates -> rank/dedupe -> summarize -> budget check -> package -> publish

Artifact	Description	Verification
Context schemas	Typed request, evidence, package, and budget definitions	Importable by all agents
Retrieval adapters	Connectors for memory and repository sources	Integration tests against fixtures
Compression policies	Role-specific prompt shaping and token rules	Versioned and unit-tested
Provenance mapper	Source-to-package traceability map	Every package includes source ids
Cache layer	Repeat-request acceleration	Invalidates correctly on source change
Telemetry hooks	Metrics and tracing for retrieval and budget health	Visible in logs and dashboards

1. Purpose

Context compression exists to prevent the agent from trying to carry the entire repository and every past conversation into the model at once. That is how you get bloated prompts, missed signals, and a system that looks informed while quietly forgetting the one file that matters.

The subsystem converts large, noisy, heterogeneous source material into compact, traceable context packets that fit the active token budget for the current task. It preserves the source provenance so later agents can inspect, verify, and rehydrate the original evidence when needed.

Problems solved:

- Prompt overflow when the repository is large or the task spans many files.
- Loss of signal when raw files are copied blindly into the prompt.
- Repeated rereading of the same documents and files across agent turns.
- Weak handoffs between research, planning, coding, and review stages.

Dependencies: the planner needs concise task-relevant context, the coding agent needs exact local evidence, the testing and review agents need recent implementation state, and the evaluator needs a compact record of what was actually asked versus what was supplied.

2. Scope

Included in this part:

- Context request classification and budget estimation.
- Retrieval from repository intelligence, code knowledge graph, file summaries, and memory stores.
- Ranked candidate selection, deduplication, and freshness filtering.
- Compression methods: extractive summaries, structured briefs, and token-aware truncation.
- Provenance tracking and context packet assembly.
- Cache management for repeated retrieval within the same task or session.
- Validation of the final package against token and coverage limits.

Excluded from this part:

- Long-term memory policy design beyond the interfaces already defined in the memory part.
- Code knowledge graph construction itself.
- Agent-specific reasoning policies for planning, coding, or evaluation.
- Model training or fine-tuning for summarization.

3. System Overview

The subsystem operates as a deterministic pipeline wrapped by LangGraph state. It does not guess what the agent needs; it gathers evidence, compresses it, and proves where every byte came from.

Core flow:

User / task request

- > Context Request Normalizer
- > Budget Estimator
- > Source Resolver
- > Retriever / Ranker
- > Summarizer / Extractor
- > Token Budget Manager
- > Provenance Packager

-> Downstream agent prompt

Key behavior:

- The same request should produce the same package unless source data or configuration changes.
- Compression must be loss-aware, not loss-free: it may shorten text, but it must not silently change meaning.
- Every compressed claim must carry a source reference, retrieval timestamp, and confidence score.

4. Internal Architecture

The architecture is organized around five responsibilities: identifying the task context, collecting candidate evidence, compressing that evidence, checking the budget, and emitting a structured package.

Component map:

Context Request Normalizer -> converts a task, turn history, and agent role into a canonical retrieval request.

Source Resolver -> selects the relevant stores: Qdrant memory, repository summaries, file-level indexes, graph queries, and cached prior packages.

Retriever / Ranker -> ranks candidate chunks by semantic similarity, graph proximity, recency, and task role relevance.

Summarizer / Extractor -> creates short form summaries for long sources while preserving explicit facts and commands.

Budget Manager -> enforces hard token ceilings and chooses what to trim, summarize, or defer.

Provenance Packager -> outputs the final context envelope with source IDs, trace links, and lineage metadata.

Interfaces required:

- request_in -> canonical context request
- candidates_in -> ranked evidence chunks
- package_out -> structured context packet
- budget_report_out -> token accounting and omission report
- cache_key -> stable key for request plus configuration

5. Project Structure

Recommended structure:

context_compression/

__init__.py

schemas.py

request_normalizer.py

source_resolver.py

retriever.py

ranker.py

summarizer.py

budget.py
 packager.py
 provenance.py
 cache.py
 policies.py
 telemetry.py
 tests/

Why each folder matters:

- schemas.py defines typed contracts shared by agents and storage layers.
- retriever.py and ranker.py isolate evidence selection from compression logic.
- summarizer.py keeps abstraction-specific summarization rules separate from generic packaging.
- budget.py is responsible only for token accounting and truncation decisions.
- packager.py assembles the final handoff object and must never call retrieval itself.
- tests/ contains golden-package tests, token-budget tests, and provenance tests.

6. Data Contracts

The subsystem exchanges structured objects, not free-form prompt blobs. This makes it testable, debuggable, and less embarrassing when someone asks why a missing file changed the answer.

Key contracts:

7. State Management

State exists at three layers:

- Working state: request-local scores, selected candidates, and token estimates live in the active LangGraph state for the current turn.
- Session state: prior packages, repeated queries, and user preference hints persist for the session and avoid repeated retrieval work.
- Project state: stable repository summaries, known module maps, and canonical source fingerprints are persisted with the project memory model.

Ownership rules:

- The request normalizer owns the canonical request object.
- The retriever owns candidate collection but not summarization.
- The budget manager owns the final inclusion/exclusion decision.
- The packager owns the emitted package and the provenance map.

Persistence rules:

- Working state is ephemeral and should be checkpointed only through LangGraph state.
- Session and project state are persisted via the memory layer and should survive process restarts.
- Cache entries must be invalidated when source fingerprints, task intent, or compression policy version changes.

8. Component Specifications

Each component below has a single job. The design goal is boring reliability, not cleverness.

A. Request Normalizer

- Purpose: translate raw user history and agent role into a canonical retrieval request.
- Inputs: task description, previous turn summaries, agent role, budget, repository metadata.
- Outputs: normalized intent, seed sources, required freshness, and coverage targets.
- Failure conditions: ambiguous intent, missing task id, or unsupported role.

B. Source Resolver

- Purpose: map a normalized request to candidate stores.
- Inputs: request, repository map, memory configuration, cache index.
- Outputs: ordered list of sources to query and priority weights.
- Failure conditions: unavailable store, invalid source fingerprint, stale cache entry.

C. Retriever / Ranker

- Purpose: fetch and rank evidence chunks.
- Inputs: source list, semantic query, graph hints, recency hints.
- Outputs: scored evidence chunks with location anchors and source provenance.
- Failure conditions: empty result set, duplicate-heavy set, or inconsistent scoring.

D. Summarizer

- Purpose: compress long evidence into short but faithful briefings.
- Inputs: selected chunks, role-specific compression policy, summary budget.
- Outputs: extractive summary, highlight bullets, omission list.
- Failure conditions: summary exceeds budget, introduces unsupported claim, or loses required commands.

E. Budget Manager

- Purpose: enforce token ceilings without hiding important evidence.
- Inputs: candidate package, reserved budget, downstream role profile.
- Outputs: accepted chunks, dropped chunks, reserved headroom, token report.
- Failure conditions: budget overflow, undersized reserved headroom, or inconsistent accounting.

F. Provenance Packager

- Purpose: create the final package for downstream agents.
- Inputs: compressed summary, selected chunks, token report, source fingerprints.
- Outputs: context package with trace map and retrieval metadata.
- Failure conditions: missing provenance, invalid ordering, or non-reproducible package id.

9. Implementation Sequence

Build order:

- Step 1: Define schemas and identifiers first. Nothing else is safe until request, evidence, and package shapes are stable.
- Step 2: Implement the request normalizer and source resolver. This establishes deterministic inputs and source selection.

- Step 3: Implement retrieval adapters for Qdrant, repository summaries, and graph queries. Without retrieval, compression has nothing to compress.
- Step 4: Add ranking and deduplication logic. This removes repeated and low-value evidence before summarization.
- Step 5: Add summarization with role-specific policies. This is where long source material becomes prompt-ready.
- Step 6: Implement budget management and trimming. It must run after summarization so it can account for the actual package cost.
- Step 7: Implement provenance packaging and cache writes. Only stable packages should be cached.
- Step 8: Wire telemetry, regression tests, and fallback behavior.

Pseudo-flow:

10. Validation Strategy

Validation must prove both correctness and restraint. A context compressor that is merely small is a failure if it drops the one file the agent needed.

Test layers:

- Unit tests for request normalization, scoring, deduplication, token accounting, and provenance generation.
- Golden tests that compare generated packages for fixed tasks and repository snapshots.
- Integration tests that run full retrieval and packaging against fixture repositories.
- Load tests that verify performance under large repositories and repeated requests.
- Regression tests that confirm cache invalidation after source changes and policy changes.

Verification criteria:

- No package may exceed the configured token budget.
- Every selected chunk must be traceable to a source id and source location.
- A repeated request with unchanged inputs should produce the same package id and equivalent package contents.
- A changed source fingerprint must force a refresh and invalidate cached packages.

11. Deliverables

Completion should produce the following artifacts:

12. Acceptance Criteria

The implementation is complete only when all of the following are true:

- Given the same request, repository snapshot, and policy version, the compressor produces a stable package id and stable provenance map.
- For every supported agent role, the compressed package stays within the configured token limit.
- Each package contains source coverage notes that explain what was included and what was intentionally omitted.
- The compressor returns a non-empty fallback package when retrieval succeeds but ranking eliminates all candidates.
- Cache entries are invalidated when source fingerprints, retrieval policy, or token budget configuration changes.
- The package can be rehydrated into the same evidence set using only its provenance map and source ids.

13. Common Failure Modes

Typical mistakes and traps:

- Summarizing before ranking, which often preserves the wrong material and wastes budget.
- Dropping provenance to save space, which makes debugging impossible.
- Treating all source types as equal and ignoring task role, recency, or graph distance.
- Compressing so aggressively that the package becomes short but useless.
- Caching without invalidation, which creates stale context that looks authoritative.
- Allowing the budget manager to trim provenance or required task instructions.

The key engineering rule is simple: never sacrifice traceability to save tokens. That is how systems become fast and wrong.

14. Future Integration

This part connects directly to later coordination and measurement stages.

- Part 12 Multi-Agent Coordination will consume compressed packages as the shared handoff format between agents.
- Part 13 Benchmark Harness will measure package quality indirectly through downstream task success, token efficiency, and retry frequency.
- The planning, coding, testing, review, evaluator, and recovery agents will all depend on this part to avoid prompt bloat.
- Future memory improvements will use the provenance map to decide which summaries should be promoted to project memory.

Design implication: the package format must remain stable and backwards-compatible once other agents depend on it.

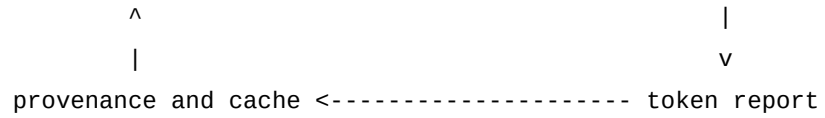
15. Completion Checklist

Before marking this part complete, verify the following:

- Schemas exist and are shared by all consumers.
- Retrieval, ranking, summarization, budget enforcement, and provenance are independently testable.
- Cache invalidation behaves correctly for source and policy changes.
- Token budget enforcement is deterministic and covered by tests.
- Every output package contains source traceability.
- Golden tests pass for representative tasks.
- Performance is acceptable on large repositories.
- Telemetry is emitted for retrieval quality and budget usage.

Appendix A. Context Package Diagram

Input request -> normalize -> resolve sources -> retrieve -> rank -> summarize -> budget
 check -> package -> handoff



Appendix B. Example Package Envelope

```

{
  package_id: contextpkg_20260603_001,
  task_id: task_1042,
  budget_used: 6420,
  selected_chunks: [file summaries, graph paths, memory notes],
  omissions: [low-score duplicates, stale docs],
  provenance_map: [...],
  freshness_window: 30d
}
  
```

End of specification.